Jobsheet 5

Langkah-langkah

Dosen Pengampu:

Mungki Astiningrum, ST, M.Kom

Disusun oleh:

Mukhammad Raffi Zabra

T1 – 1G

254107020059

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur

65141

TAHUN 2025-2026

5.2 Menghitung Nilai Faktorial dengan Algoritma Brute Force dan Divide and Conquer
5.2.1. Langkah-langkah Percobaan

Code Faktorial:

```
package BruteForceDivideConquer.minggu5;

public class Faktorial {
    int faktorialBF(int n){
        int fakto = 1;
        for(int i=1;i<=n;i++){
            fakto = fakto*i;
        }
        return fakto;
    }

    int faktorialDC(int n){
        if(n==1){
            return 1;
        }else{
            int fakto = n * faktorialBF(n-1);
            return fakto;
        }
    }
}
```

Code Faktorialmain:

```
package BruteForceDivideConquer.minggu5;

import java.util.Scanner;
public class Faktorialmain {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan nilai : ");
        int nilai = input.nextInt();

        Faktorial fk = new  Faktorial();
        System.out.println("Nilai faktorial "+nilai+
        " menggunakan BF: "+fk.faktorialBF(nilai));
        System.out.println("Nilai faktorial "+nilai+
        " menggunakan DC: "+fk.faktorialDC(nilai));
    }
}
```

Hasil :

```
Masukkan nilai : 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
PS C:\Users\mukha\OneDrive\Documents\Semerter 2\Praktikum Algoritma Struktur Data>
```

Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!

- Bagian if digunakan sebagai base case atau kondisi dasar rekursi
- Saat n == 1, proses berhenti dan method mengembalikan nilai 1
- Bagian else digunakan untuk proses rekursif, yaitu memecah masalah menjadi bentuk yang lebih kecil, n * faktorialDC(n - 1)

2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!

- **Saya coba pakai while**

Code faktorial:

```
package BruteForceDivideConquer.minggu5;
public class Faktorial {
int faktorialBF(int n){
int fakto = 1;
int i = 1;
while(i<=n){
fakto = fakto *i;
i++;
}
return fakto;
}
int faktorialDC(int n){
if(n==1){
return 1;
}else{
int fakto = n * faktorialBF(n-1);
return fakto;
}
}
}
```

Code faktorialmain:

```
package BruteForceDivideConquer.minggu5;
import java.util.Scanner;
public class Faktorialmain {
public static void main(String[] args) {
Scanner input = new Scanner(System.in);
System.out.print("Masukkan nilai : ");
int nilai = input.nextInt();
Faktorial fk = new Faktorial();
System.out.println("Nilai faktorial "+nilai+
" menggunakan BF: "+fk.faktorialBF(nilai));
System.out.println("Nilai faktorial "+nilai+
" menggunakan DC: "+fk.faktorialDC(nilai));
}
}
```

Hasil :

```
Masukkan nilai : 5
Nilai faktorial 5 menggunakan BF: 120
Nilai faktorial 5 menggunakan DC: 120
PS C:\Users\mukha\OneDrive\Documents\Semerter 2\Praktikum Algoritma Struktur Data>
```

Kesimpulan :

Method faktorialBF() dapat dibuat dengan jenis perulangan lain selain for, selama logika perulangannya tetap benar

3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1); !

- Yang fakto*=i; = digunakan untuk metode brute force
- Yang int fakto = n * faktorialDC(n-1); = digunakan untuk metode divide and conquer

4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!

- Method faktorialBF() menghitung faktorial menggunakan perulangan dari 1 sampai n. Sedangkan faktorialDC() menghitung faktorial menggunakan rekursi dengan memanggil fungsi dirinya sendiri sampai mencapai kondisi dasar (n == 1).
  Jadi, perbedaannya adalah faktorialBF() menggunakan loop, sedangkan faktorialDC() menggunakan rekursi

5.3 Menghitung Hasil Pangkat dengan Algoritma Brute Force dan Divide and Conquer
5.3.1. Langkah-langkah Percobaan

Code pangkat:

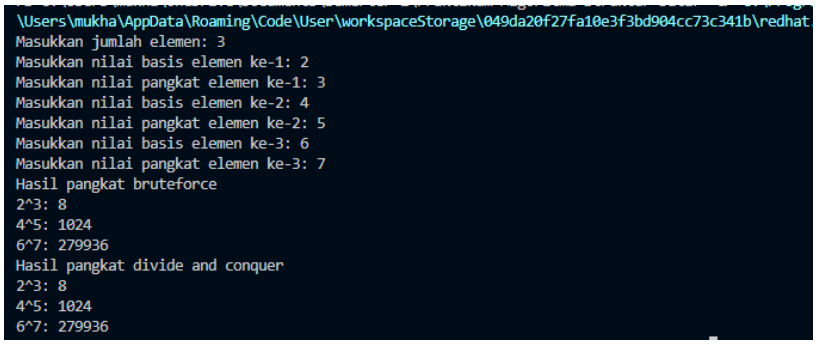
```
package BruteForceDivideConquer.minggu5;

public class pangkat {
   int nilai,pangkat;

   pangkat(int n, int p){
      nilai = n;
      pangkat = p;
   }

   int pangkatBF(int a, int n){
      int hasil = 1;
      for(int i=0; i<n; i++){
         hasil = hasil*a;
      }
      return  hasil;
   }

   int pangkatDC(int a, int n){
      if(n==1){
         return a;
      }else{
         if(n%2==1){
            return  (pangkatDC(a, n/2)*pangkatDC(a, n/2)*a);
         }else{
            return (pangkatDC(a, n/2)*pangkatDC(a, n/2));
         }
      }
   }

}
```

Code pangkatmain:

```
package BruteForceDivideConquer.minggu5;

import java.util.Scanner;

public class pangkatmain {
   public static void main(String[] args) {
      Scanner input = new Scanner(System.in);
      System.out.print("Masukkan jumlah elemen: ");
      int elemen = input.nextInt();

      pangkat[] png = new pangkat[elemen];
      for(int i=0; i<elemen;i++){
         System.out.print("Masukkan nilai basis elemen ke-"+(i+1)+": ");
```

```
            int basis = input.nextInt();
            System.out.print("Masukkan nilai pangkat elemen ke-"+(i+1)+": ");
            int pangkat = input.nextInt();
            png[i] = new pangkat(basis, pangkat);
        }

        System.out.println("Hasil pangkat bruteforce");
        for(pangkat p : png){
            System.out.println(p.nilai+"^"+p.pangkat+": "+p.pangkatBF(p.nilai, p.pangkat));
        }
        System.out.println("Hasil pangkat divide and conquer");
        for(pangkat p : png){
            System.out.println(p.nilai+"^"+p.pangkat+": "+p.pangkatDC(p.nilai, p.pangkat));
        }
    }
}
```

Hasil :



```
\Users\mukha\AppData\Roaming\Code\User\workspaceStorage\049da20f27fa10e3f3bd904cc73c341b\redhat
Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1: 2
Masukkan nilai pangkat elemen ke-1: 3
Masukkan nilai basis elemen ke-2: 4
Masukkan nilai pangkat elemen ke-2: 5
Masukkan nilai basis elemen ke-3: 6
Masukkan nilai pangkat elemen ke-3: 7
Hasil pangkat bruteforce
2^3: 8
4^5: 1024
6^7: 279936
Hasil pangkat divide and conquer
2^3: 8
4^5: 1024
6^7: 279936
```

Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!

- Method pangkatBF() menghitung perpangkatan dengan perulangan, yaitu mengalikan nilai basis secara berulang sebanyak nilai pangkat.
  Sedangkan pangkatDC() menghitung perpangkatan dengan rekursi Divide and Conquer, yaitu membagi masalah menjadi pangkat yang lebih kecil lalu menggabungkan hasilnya.

- Jadi, perbedaannya:
  pangkatBF() → iteratif
  pangkatDC() → rekursif

2. Apakah tahap combine sudah termasuk dalam kode tersebut?Tunjukkan!

- Ya, tahap combine sudah ada pada method pangkatDC();

- Bagian combine : return (pangkatDC(a, n/2)*pangkatDC(a, n/2)*a); dan return (pangkatDC(a, n/2)*pangkatDC(a, n/2));

3. Pada method pangkatBF()terdapat parameter untuk melewatkan nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?

- Menurut saya, method tersebut **tetap relevan** jika memakai parameter, karena method menjadi lebih fleksibel dan bisa digunakan untuk nilai apa saja tanpa bergantung pada atribut objek.
  Tetapi, method itu juga **bisa dibuat tanpa parameter** karena class sudah memiliki atribut:

- Nilai
- Pangkat
- Jadi method cukup memakai atribut yang ada di dalam objek

- Contoh pangkatBF() tanpa parameter.

Code pagkat:

```
package BruteForceDivideConquer.minggu5;

public class pangkat {
   int nilai,pangkat;

   pangkat(int n, int p){
      nilai = n;
      pangkat = p;
   }

   int pangkatBF(){
      int hasil = 1;
      for(int i = 0; i < pangkat; i++){
      hasil = hasil * nilai;
   }
   return hasil;
   }

   int pangkatDC(int a, int n){
      if(n==1){
         return a;
      }else{
         if(n%2==1){
            return  (pangkatDC(a, n/2)*pangkatDC(a, n/2)*a);
         }else{
            return (pangkatDC(a, n/2)*pangkatDC(a, n/2));
         }
      }
   }

}
```

Code pangkatmain:

```
package BruteForceDivideConquer.minggu5;

import java.util.Scanner;

public class pangkatmain {
   public static void main(String[] args) {
      Scanner input = new Scanner(System.in);
      System.out.print("Masukkan jumlah elemen: ");
      int elemen = input.nextInt();

      pangkat[] png = new pangkat[elemen];
      for(int i=0; i<elemen;i++){
         System.out.print("Masukkan nilai basis elemen ke-"+(i+1)+": ");
         int basis = input.nextInt();
         System.out.print("Masukkan nilai pangkat elemen ke-"+(i+1)+": ");
         int pangkat = input.nextInt();
         png[i] = new pangkat(basis, pangkat);
      }
```

```
        System.out.println("Hasil pangkat bruteforce");
        for(pangkat p : png){
            System.out.println(p.nilai+"^"+p.pangkat+": "+p.pangkatBF());
        }
        System.out.println("Hasil pangkat divide and conquer");
        for(pangkat p : png){
            System.out.println(p.nilai+"^"+p.pangkat+": "+p.pangkatDC(p.nilai, p.pangkat));
        }
    }
}
```

Hasil :

```
um Algoritma Struktur Data_f8e480c\bin' 'BruteForceDivideConquer.minggu5.
Masukkan jumlah elemen: 3
Masukkan nilai basis elemen ke-1: 2
Masukkan nilai pangkat elemen ke-1: 3
Masukkan nilai basis elemen ke-2: 4
Masukkan nilai pangkat elemen ke-2: 5
Masukkan nilai basis elemen ke-3: 6
Masukkan nilai pangkat elemen ke-3: 7
Hasil pangkat bruteforce
2^3: 8
4^5: 1024
6^7: 279936
Hasil pangkat divide and conquer
2^3: 8
4^5: 1024
6^7: 279936
```

4. Tarik tentang cara kerja method pangkatBF() dan pangkatDC()!

- Method pangkatBF() bekerja dengan cara mengalikan basis secara berulang menggunakan perulangan sampai jumlah pangkat terpenuhi.
  Sedangkan method pangkatDC() bekerja dengan cara memecah masalah perpangkatan menjadi bagian yang lebih kecil menggunakan rekursi, lalu menggabungkan hasilnya kembali.

- Jadi:
  pangkatBF() lebih sederhana karena memakai loop
  pangkatDC() lebih sesuai dengan konsep Divide and Conquer karena memakai rekursi dan combine

5.4 Menghitung Sum Array dengan Algoritma Brute Force dan Divide and Conquer
5.4.1. Langkah-langkah Percobaan

Code sum:

```
package BruteForceDivideConquer.minggu5;

public class sum {
    double keuntungan[];

    sum(int el){
        keuntungan = new double[el];
    }

    double totalBF(){
        double total=0;
        for(int i=0;i<keuntungan.length;i++){
            total = total+keuntungan[i];
        }
        return total;
    }

    double totalDC(double  arr[], int l, int r){
```

```
        if(l==r){
            return  arr[l];
        }

        int mid = (l+r)/2;
        double lsum = totalDC(arr, l, mid);
        double rsum = totalDC(arr, mid+1, r);
        return  lsum+rsum;
    }

}
```

Code summain:

```
package BruteForceDivideConquer.minggu5;

import java.util.Scanner;

public class summain {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();

        sum sm = new sum(elemen);
        for(int i=0; i<elemen;i++){
            System.out.print("Masukkan keuntungan ke-"+(i+1)+": ");
            sm.keuntungan[i] = input.nextDouble();
        }

        System.out.println("Total keuntungan menggunakan BruteForce: "+sm.totalBF());
        System.out.println("Total keuntungan menggunakan Divide and Conquer:
"+sm.totalDC(sm.keuntungan,0,elemen-1));
    }

}
```

Hasil :

```
r.minggu5.summain'
Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 10
Masukkan keuntungan ke-2: 20
Masukkan keuntungan ke-3: 30
Masukkan keuntungan ke-4: 40
Masukkan keuntungan ke-5: 50
Total keuntungan menggunakan BruteForce: 150.0
Total keuntungan menggunakan Divide and Conquer: 150.0
PS C:\Users\mukha\OneDrive\Documents\Semerter 2\Praktikum Algoritma Struktur Data>
```

Pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?

- digunakan untuk membagi array menjadi dua bagian, yaitu bagian kiri dan bagian kanan. Pembagian ini diperlukan agar algoritma Divide and Conquer dapat memecah masalah menjadi submasalah yang lebih kecil

2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```
double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);
```

- Statement tersebut digunakan untuk memanggil method secara rekursif pada dua bagian array:

- lsum menghitung total bagian kiri

- rsum menghitung total bagian kanan

Dengan begini array diproses secara bertahap sampai mencapai kondisi dasar

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```
return lsum+rsum;
```

- Penjumlahan ini diperlukan untuk menggabungkan hasil perhitungan dari dua bagian array kiri dan kanan sehingga menghasilkan total keseluruhan nilai pada array

4. Apakah base case dari totalDC()?

- if(l==r){
  return arr[l];
  }

Jadi jika indeks kiri sama dengan kanan, berarti hanya ada satu elemen sehingga langsung mengembalikan nilai elemen tersebut

5. Tarik Kesimpulan tentang cara kerja totalDC()

- Method totalDC() bekerja dengan cara membagi array menjadi dua bagian secara rekursif, kemudian menghitung total masing-masing bagian dan menggabungkannya kembali hingga mendapatkan total keseluruhan nilai array